

WFLOW-01: Workflow Fundamentals

Series: WFLOW — Workflows and Alert Notifications | **Notebook:** 1 of 10 | **Created:** January 2026 | **Last Updated:** 09/28/2026

Introduction to Dynatrace Workflows

Dynatrace Workflows is the automation engine that enables event-driven automation, scheduled tasks, and integration orchestration. This notebook introduces core concepts, components, and your first workflow.

Table of Contents

- [1. What Are Workflows?](#)
 - [2. Workflows vs Other Automation](#)
 - [3. Workflow Components](#)
 - [4. Accessing Workflows](#)
 - [5. Execution Model](#)
 - [6. Your First Workflow](#)
 - [7. Viewing Execution History](#)
-

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Platform subscription

Requirement	Details
Permissions	<code>automation:workflows:read</code> , <code>automation:workflows:write</code>
Prior Knowledge	Basic Dynatrace navigation

1. What Are Workflows?

Workflows are automated sequences of tasks that execute in response to events, schedules, or manual triggers. They enable:

Capability	Example
Alert Notifications	Send Slack/Teams messages when problems occur
Incident Management	Create PagerDuty/ServiceNow tickets automatically
Auto-Remediation	Restart pods, scale resources, run scripts
Reporting	Generate and email daily reports
Integration	Sync data with external systems
Data Enrichment	Query APIs and enrich events

Key Benefits

- **No Infrastructure** - Runs in Dynatrace, no external servers needed
- **Event-Driven** - Triggers on detected problems, metric events, schedules
- **Low-Code + Code** - Visual builder with JavaScript option
- **Secure** - Built-in secrets management, RBAC, audit logs
- **Observable** - Execution history, metrics, debugging

2. Workflows vs Other Automation

Dynatrace offers multiple automation approaches. When should you use Workflows?

Approach	Best For	Limitations
Workflows	Event-driven automation, notifications, integrations	Rate limits, execution time limits
Alerting Profiles (Legacy)	Simple email/webhook notifications	Limited routing, no logic
Apps (AppEngine)	Custom UI, complex applications	Requires development expertise
Extensions	Data collection, custom metrics	Not for automation/notifications
Site Reliability Guardian	Release validation, SLO verification	Specific to deployment validation

Migration from Legacy Alerting

Legacy Alerting	Workflow Equivalent
Problem notification → Email	Detected Problem trigger → Email task
Problem notification → Webhook	Detected Problem trigger → HTTP Request task
Custom integration	Detected Problem trigger → JavaScript + HTTP

Recommendation: New implementations should use Workflows. Alerting profiles are labeled **Dynatrace Classic** and Dynatrace recommends simple workflows for new setups — but no deprecation or end-of-life date has been published, so existing profiles keep working. The one real forcing function is Management Zones: a profile scoped by an MZ has no successor filter in the alerting model and must be rebuilt as a problem-triggered workflow before those zones are retired (MZ2POL-01 §5).

3. Workflow Components

Every workflow consists of these components:

3.1 Trigger

What starts the workflow:

Trigger Type	Starts When	Use Case
Detected Problem	Dynatrace Intelligence detects a problem	Alert notifications
Davis Event	An anomaly detector raised a Davis event (per-alert, before grouping into a problem)	Per-alert automation
Schedule	Cron expression matches	Daily reports
On-Demand	Manual execution or API call	Testing, ad-hoc runs
Event Trigger	Custom/business event ingested	Business process automation

3.2 Tasks

Actions the workflow performs:

Task Category	Examples
Notifications	Slack, Teams, Email, PagerDuty
Incident Management	ServiceNow, Jira
Data Queries	DQL queries, entity lookups
HTTP Requests	REST API calls to any endpoint
JavaScript	Custom code execution
Control Flow	Wait, conditions, loops

3.3 Conditions

Logic that controls task execution:

```
conditions:  
  - name: is_availability_problem  
    expression: '{{ event()["event.category"] == "AVAILABILITY" }}'
```

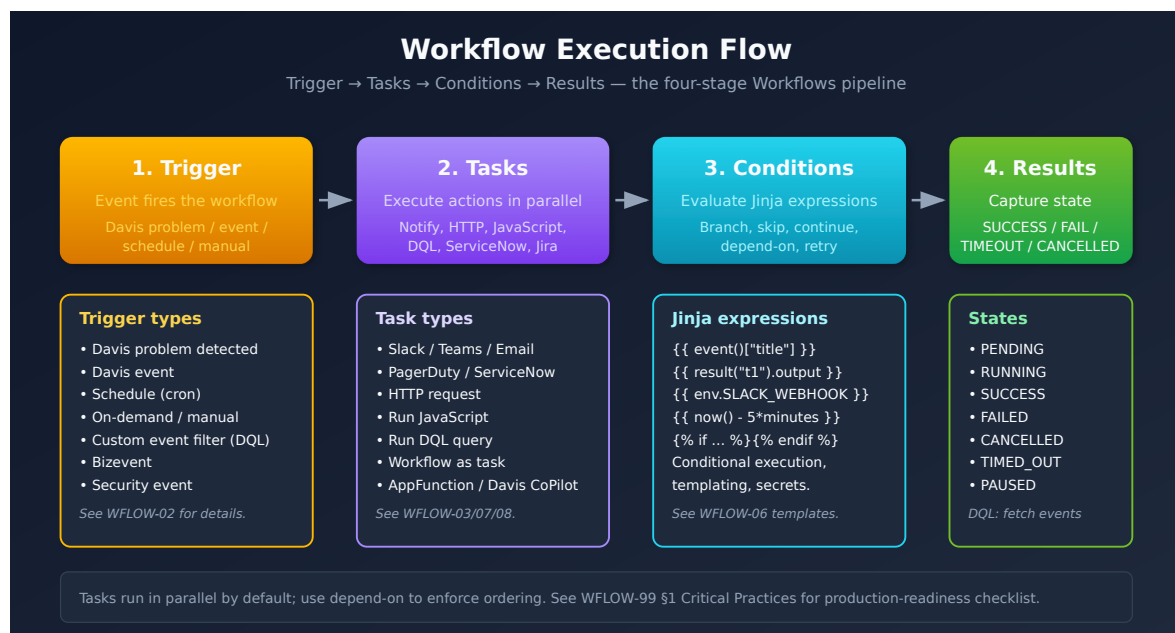
3.4 Expressions

Dynamic values using Jinja2 syntax:

```
{{ event()["event.name"] }}      # Access trigger data (problem title)
{{ problem_link() }}            # Link to the problem (Problem trigger only)
{{ result("task_name") }}       # Previous task result – shape depends on
task type; see WFLOW-08 §6
{{ input("environment") }}      # Workflow input
```

There is no `env` object in the expression language. Secrets belong in a **connection** or the **Credential Vault**, never in an expression (WFLOW-09 §2).

Visual: Workflow Execution Flow



4. Accessing Workflows

Navigation

1. Open Dynatrace
2. Go to **Apps** in the left navigation
3. Search for and open **Workflows**

Or use the direct URL: `https://<your-environment>/ui/apps/dynatrace.automations/workflows`

Required Permissions

Permission	Allows
<code>automation:workflows:read</code>	View workflows, view execution history
<code>automation:workflows:write</code>	Create, edit, delete workflows
<code>automation:workflows:run</code>	Execute workflows manually
<code>automation:workflows:admin</code>	Full admin access

Workflow Listing

The Workflows app shows:

- **My Workflows** - Workflows you created
- **Shared with me** - Workflows shared by others
- **All Workflows** - All workflows you can access

5. Execution Model

Understanding how workflows execute is important for designing reliable automation.

Execution Flow

```
Trigger fires
↓
Workflow instance created
↓
Tasks execute (sequential or parallel)
↓
Conditions evaluated at each step
↓
Results captured
↓
Execution complete (success/failure)
```

Platform Limits

Two distinct timeouts apply to every workflow task — confusing them is one of the most common causes of "why did my task fail at exactly 2 minutes?":

Limit	Default	Maximum	Scope
Task timeout	60 minutes	7 days	Wall-clock budget for the whole task, including retries and loops. Configured per task via the <code>timeout</code> field (in seconds).
Dynatrace runtime timeout	120 seconds	—	Per-action execution budget inside the AutomationEngine runtime (applies to DQL queries and individual JavaScript/HTTP calls). Hits the action, not the task — the task itself keeps going.
DQL <code>requestTimeoutMilliseconds</code>	No default documented	—	The JavaScript SDK's <code>queryExecute()</code> parameter — how long the call waits for the result before returning a <code>requestToken</code> instead, in milliseconds. It does not stop the query; poll with <code>queryPoll()</code> to collect the result (WFLOW-08 §2).

Why this matters. Raising the task `timeout` does not help a DQL query that's hitting the 120-second runtime budget — you need to narrow the query window, pre-aggregate, or split the work. Conversely, an approval task waiting for a human pager-out doesn't need a runtime budget — it needs a long task timeout (`timeout: 1800` for 30 minutes, `timeout: 86400` for 24 hours).

Concrete handling: see **WFLOW-08 § Configuring Task Timeouts** for the YAML/JSON shape on `execute-dql-query`, `run-javascript`, and approval tasks.

Other platform constraints. Concurrency caps, per-workflow task counts, and rate limits exist but are not consistently published in current Dynatrace docs and have changed over the product's life. In community practice, treat "keep workflows small (under ~20 tasks), keep them focused (one trigger → one outcome), and don't fan out hundreds of concurrent executions" as the operational rule. Verify against your tenant's actual behavior under load before designing around a specific number.

Execution States

State	Meaning
<code>RUNNING</code>	Currently executing
<code>SUCCESS</code>	Completed successfully
<code>ERROR</code>	A task failed or timed out — there is no separate <code>TIMED_OUT</code> state
<code>CANCELLED</code>	Manually cancelled
<code>SKIPPED</code> / <code>DISCARDED</code>	Task level only — the task's conditions were not met

These are the values `dt.automation_engine.state` carries in `dt.system.events` (the queries in §7 read them); `FAILED`, `SUCCEEDED` and `TIMED_OUT` are not values.

6. Your First Workflow

Let's create a simple workflow that logs a message when manually triggered.

Step 1: Create New Workflow

1. Open **Workflows** app
2. Click **+ Workflow**
3. Name it: `Hello World workflow`

Step 2: Configure Trigger

1. Click the trigger box (top of canvas)
2. Select **On demand** trigger
3. This allows manual execution for testing

Step 3: Add a Task

1. Click **+** below the trigger
2. Search for **Run JavaScript**
3. Add the following code:

```
export default async function() {
  const now = new Date().toISOString();
  console.log(`Hello from Dynatrace Workflows at ${now}`);

  return {
    message: "Workflow executed successfully!",
    timestamp: now
  };
}
```

Step 4: Save and Run

1. Click **Save**
2. Click **Run** (play button)
3. View execution results

Expected Output

The task output should show:

```
{
  "message": "Workflow executed successfully!",
  "timestamp": "2026-01-27T12:00:00.000Z"
}
```

7. Viewing Execution History

Monitor your workflow executions using DQL queries.

```
// Recent workflow executions
// Data object corrected 08/12/2026. Workflow executions are NOT in `events`,
// and there is no
// `automation.task.execution` / `automation.workflow.execution` event type in
// any spelling – those
// filters matched nothing, silently, in 25 cells. AutomationEngine writes to
// `dt.system.events` with
// `event.kind == "WORKFLOW_EVENT"` (5.7M records / 30d here), split by
// `event.type`:
//   WORKFLOW_EXECUTION · TASK_EXECUTION · ACTION_EXECUTION ·
//   WORKFLOW_CREATED/UPDATED/DELETED
// Fields are the `dt.automation_engine.*` family:
//   .workflow.title / .workflow.id      (was workflow.name)
//   .task.name                          (was task.name)
//   .action.function / .action.app      (was task.type – e.g. run-javascript,
//   send-email,
//                                       snow-search-incidents)
//   .state                             (was task.status / execution.status)
//                                       values RUNNING · SUCCESS · ERROR ·
//   DISCARDED · SKIPPED
//                                       – note "FAILED" is NOT a value; it
//   is ERROR
//   .state.is_final                    true only on terminal records –
//   filter on it, otherwise a
//                                       single execution is counted once as
//   RUNNING and again as
//                                       SUCCESS/ERROR
//   .state_info                        (was task.error) · duration (was
//   task.duration)
// Enumerate with:
//   fetch dt.system.events, from:-24h | filter event.kind == "WORKFLOW_EVENT"
//   | limit 1
//   fetch dt.system.events, from:-24h
//   | filter event.kind == "WORKFLOW_EVENT"
//   | filter event.type == "WORKFLOW_EXECUTION"
//   | filter dt.automation_engine.state.is_final == true
//   | fields timestamp, dt.automation_engine.workflow.title,
//   dt.automation_engine.state, duration
//   | sort timestamp desc
//   | limit 25
```

```

// Workflow execution summary
// Data object corrected 08/12/2026. Workflow executions are NOT in `events`,
and there is no
// `automation.task.execution` / `automation.workflow.execution` event type in
any spelling – those
// filters matched nothing, silently, in 25 cells. AutomationEngine writes to
`dt.system.events` with
// `event.kind == "WORKFLOW_EVENT"` (5.7M records / 30d here), split by
`event.type`:
//   WORKFLOW_EXECUTION · TASK_EXECUTION · ACTION_EXECUTION ·
WORKFLOW_CREATED/UPDATED/DELETED
// Fields are the `dt.automation_engine.*` family:
//   .workflow.title / .workflow.id      (was workflow.name)
//   .task.name                          (was task.name)
//   .action.function / .action.app      (was task.type – e.g. run-javascript,
send-email,
//                                       snow-search-incidents)
//   .state                             (was task.status / execution.status)
//                                       values RUNNING · SUCCESS · ERROR ·
DISCARDED · SKIPPED
//                                       – note "FAILED" is NOT a value; it
is ERROR
//   .state.is_final                    true only on terminal records –
filter on it, otherwise a
//                                       single execution is counted once as
RUNNING and again as
//                                       SUCCESS/ERROR
//   .state_info                        (was task.error) · duration (was
task.duration)
// Enumerate with:
//   fetch dt.system.events, from:-24h | filter event.kind == "WORKFLOW_EVENT"
| limit 1
fetch dt.system.events, from:-7d
| filter event.kind == "WORKFLOW_EVENT"
| filter event.type == "WORKFLOW_EXECUTION"
| filter dt.automation_engine.state.is_final == true
| summarize executions = count(), by:{dt.automation_engine.workflow.title,
dt.automation_engine.state}
| sort executions desc
| limit 25

```

```
// Failed workflow executions
// Data object corrected 08/12/2026. Workflow executions are NOT in `events`,
and there is no
// `automation.task.execution` / `automation.workflow.execution` event type in
any spelling – those
// filters matched nothing, silently, in 25 cells. AutomationEngine writes to
`dt.system.events` with
// `event.kind == "WORKFLOW_EVENT"` (5.7M records / 30d here), split by
`event.type`:
//   WORKFLOW_EXECUTION · TASK_EXECUTION · ACTION_EXECUTION ·
WORKFLOW_CREATED/UPDATED/DELETED
// Fields are the `dt.automation_engine.*` family:
//   .workflow.title / .workflow.id      (was workflow.name)
//   .task.name                          (was task.name)
//   .action.function / .action.app      (was task.type – e.g. run-javascript,
send-email,
//                                       snow-search-incidents)
//   .state                              (was task.status / execution.status)
//                                       values RUNNING · SUCCESS · ERROR ·
DISCARDED · SKIPPED
//                                       – note "FAILED" is NOT a value; it
is ERROR
//   .state.is_final                     true only on terminal records –
filter on it, otherwise a
//                                       single execution is counted once as
RUNNING and again as
//                                       SUCCESS/ERROR
//   .state_info                         (was task.error) · duration (was
task.duration)
// Enumerate with:
//   fetch dt.system.events, from:-24h | filter event.kind == "WORKFLOW_EVENT"
| limit 1
fetch dt.system.events, from:-24h
| filter event.kind == "WORKFLOW_EVENT"
| filter event.type == "WORKFLOW_EXECUTION"
| filter dt.automation_engine.state == "ERROR"
| fields timestamp, dt.automation_engine.workflow.title,
dt.automation_engine.state_info
| sort timestamp desc
| limit 25
```

Next Steps

Now that you understand workflow fundamentals, continue with:

Recommended Path

1. **WFLOW-02: Triggers & Event Types** - Configure event-driven triggers
2. **WFLOW-03: Alert Notification Basics** - Send Slack, Teams, email alerts
3. **WFLOW-04: Advanced Notification Routing** - Conditional routing and escalation

Key Takeaways

- Workflows automate responses to events, schedules, and manual triggers
 - Components: Triggers → Tasks → Conditions → Results
 - Use Workflows for notifications, integrations, and auto-remediation
 - Monitor execution history via DQL queries
-

Summary

In this notebook, you learned:

- What Dynatrace Workflows are and when to use them
 - How workflows compare to other automation options
 - Core workflow components (triggers, tasks, conditions)
 - How to access and navigate the Workflows app
 - The execution model and limits
 - How to create your first workflow
 - How to query execution history
-

References

- [Workflows umbrella \(DT docs\)](#)
- [Workflow triggers \(DT docs\)](#)
- [Workflow actions \(DT docs\)](#)
- [Workflow reference / Jinja expressions \(DT docs\)](#)
- [Alerting and notifications umbrella \(DT docs\)](#)

- [Davis Problems app \(DT docs\)](#)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.